



Machine Learning and Neural Networks

DeepPumas Workshop

Authors: Andreu Vall, Mohamed Tarek, and Niklas Korsbo

Pumas-AI Inc

Jun 2, 2025



Broad types of machine learning

- Supervised machine learning
 - Prediction
 - Classification
- Unsupervised machine learning
 - Generative models
 - Feature extraction
 - Clustering
- Semi-supervised machine learning
 - Has components of both supervised and unsupervised learning
 - Examples:
 - Semi-supervised conditional generative models
 - Feature extraction followed by supervised ML

For depth here,
check out our AIDD course.

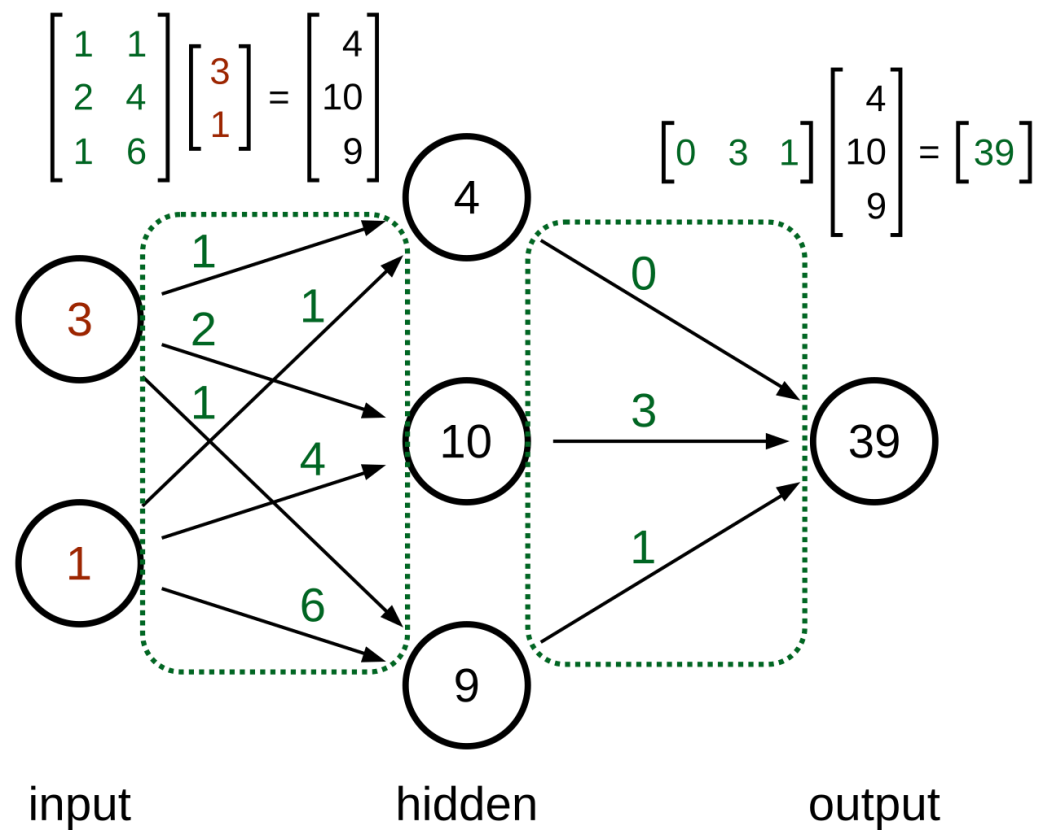


Fundamental Concepts of Machine Learning and Neural Networks



Neural Networks

f = g = identity



Training

- Find network parameters $\mathbf{W}$ that minimize a suitable cost function

$$R_{\text{emp}}(\mathbf{W}, \mathbf{X}, \mathbf{Y}) = \frac{1}{N} \sum_{n=1}^N L(y^n, \hat{y}^n(\mathbf{x}^n; \mathbf{W}))$$

- Iteratively refine the parameters by gradient-based optimization

$$\mathbf{W}^{\text{new}} = \mathbf{W}^{\text{old}} - \eta \nabla_{\mathbf{W}} R_{\text{emp}}(\mathbf{W}, \mathbf{X}, \mathbf{Y})$$



Control on Withheld Data

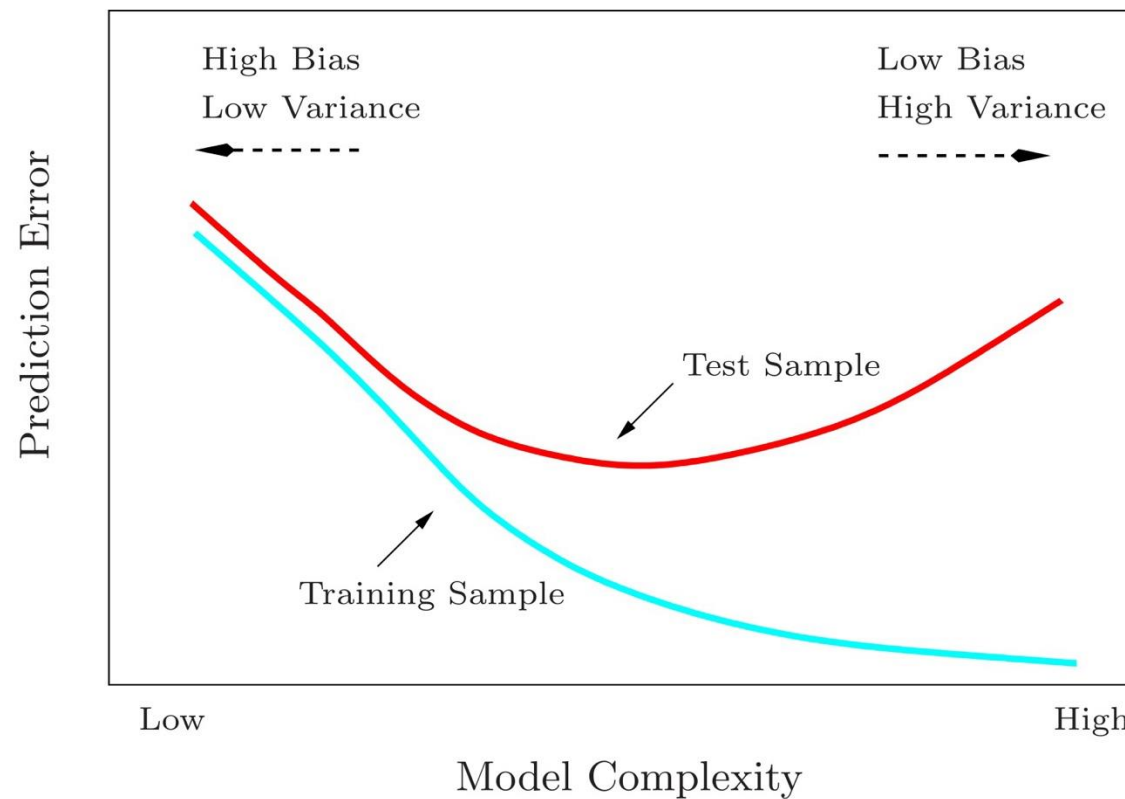


FIGURE 2.11. *Test and training error as a function of model complexity.*

Figure by [Hastie et al.](#), 2008

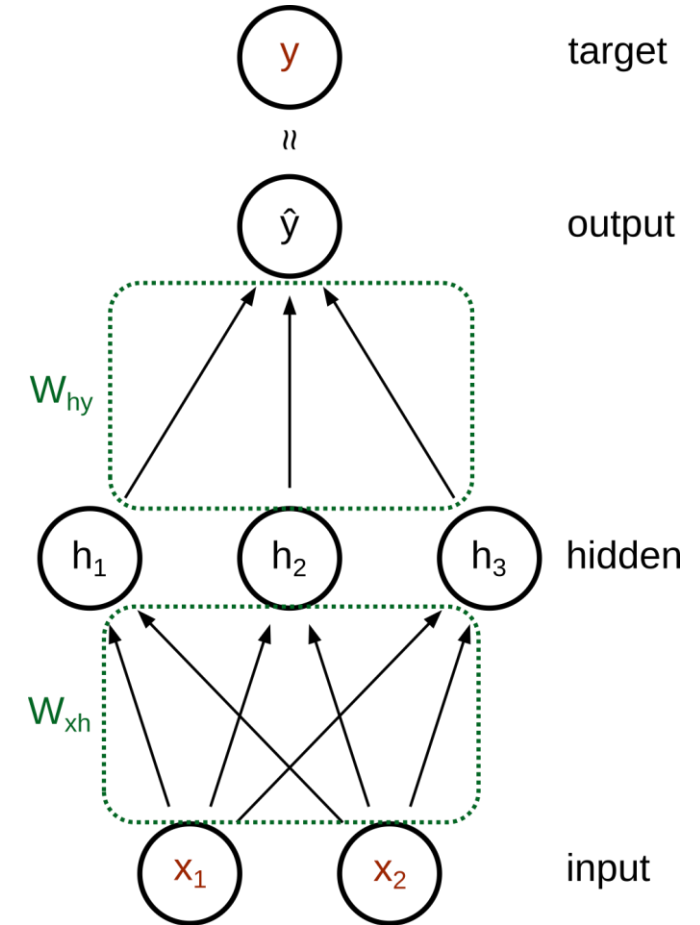


Practical Guide to Training Neural Networks



Feed-Forward Networks

- The basic interface is quite given by the task
 - Number of input units
 - Number of output units
 - Activation function for the output layer
- The *architecture* must be selected
 - Number of hidden layers
 - Number of units per hidden layer
 - Activation function for the hidden layers
 - Parameters of the optimization algorithm





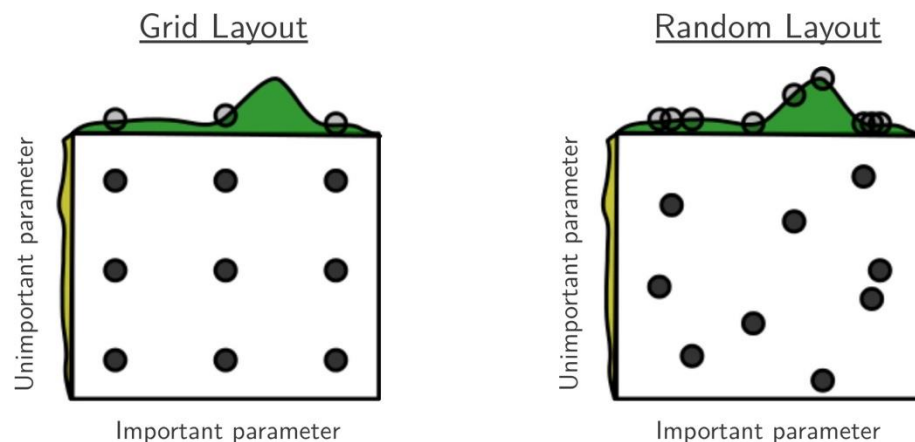
Activation Functions

- At the output layer we typically use
 - the identity function for regression tasks,
 - the sigmoid function for binary classification tasks,
 - the softmax function for multiclass classification tasks, and
 - the softplus function to ensure non-negativity.
- At the hidden layers,
 - traditional activations are the sigmoid and tanh functions,
 - but deeper networks will suffer from vanishing gradients.
 - In this case, experiment with the ReLU and SELU functions.



Architecture Selection

- Typically based on empirical investigation and assessment on withheld data
- Initial exploration to get a feel of reasonable network dimensions
- Hyperparameter search
 - Design a grid of hyperparameters based on the previous exploration
 - In case of a tight budget, pay attention to the learning rate
 - Exhaustive, random, Bayesian search



Bergstra and Bengio “Random search for hyper-parameter optimization.” ([JMLR](#), 2012)



Underfitting and Overfitting

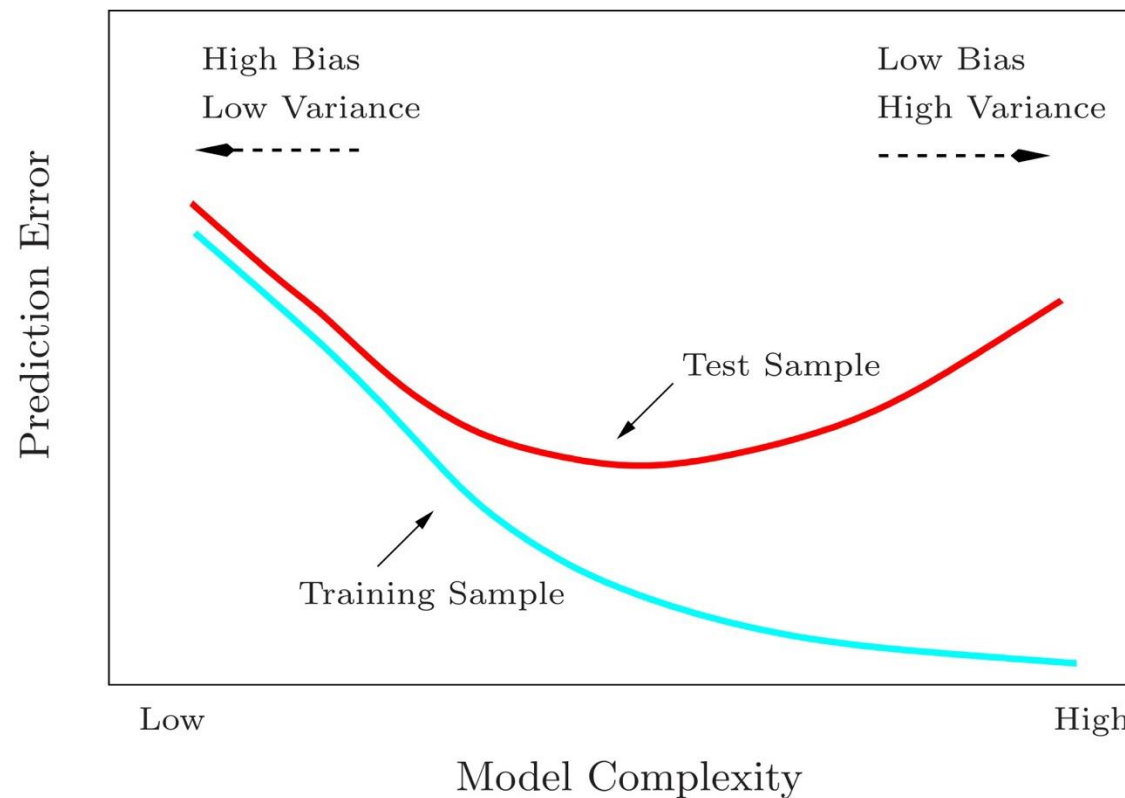


FIGURE 2.11. *Test and training error as a function of model complexity.*

Figure by [Hastie et al.](#), 2008



Strategies to Prevent Overfitting

- Early stopping based on withheld data
- Most careful data splitting to avoid data leakage
- Networks as large as possible, but not larger
- Larger and more diverse training datasets
- Regularization
 - Penalization of the cost function
 - Dropout (Srivastava et al., 2014)
 - Data *augmentation*



Penalization of the Cost Function

- Cost function

$$\arg \min_{\mathbf{W}} \frac{1}{N} \sum_{n=1}^N L(y^n, \hat{y}^n(\mathbf{x}^n; \mathbf{W}))$$

- Penalized cost function reduces expressiveness

$$\arg \min_{\mathbf{W}} \frac{1}{N} \sum_{n=1}^N L(y^n, \hat{y}^n(\mathbf{x}^n; \mathbf{W})) + \lambda \|\mathbf{W}\|_p$$

- Typically using $L1$ or $L2$ norm



Hands-on session: Fitting, Overfitting and Regularizing Neural Networks



Conditional generative models



Generative models

- Definitions
 - $\mathbf{z}$: latent variables of dimension d
 - $\mathbf{y}$: observed response/data
 - $\mathbf{y}_g$: generated/simulated/synthetic response/data
- Model

$$\begin{aligned}\mathbf{y}_g &= \mathbf{f}(\mathbf{z}) + \boldsymbol{\epsilon} \\ \mathbf{z} &\sim \text{Normal}(0, \mathbf{I}_{d \times d}) \\ \epsilon_i &\sim \text{Normal}(0, \sigma^2)\end{aligned}$$

- Objective: choose $\mathbf{f}$ such that the distribution of $\mathbf{y}_g$ is close to the distribution of the observed data $\mathbf{y}$



Conditional generative models

- Definitions
 - $\mathbf{z}$: latent variables of dimension d
 - $\mathbf{x}$: observed covariates
 - $\mathbf{y}$: observed response
 - $\mathbf{y}_g$: generated/simulated/synthetic response
- Model

$$\begin{aligned}\mathbf{y}_g &= \mathbf{f}(\mathbf{z}, \mathbf{x}) + \boldsymbol{\epsilon} \\ \mathbf{z} &\sim \text{Normal}(\mathbf{0}, \mathbf{I}_{d \times d}) \\ \epsilon_i &\sim \text{Normal}(0, \sigma^2)\end{aligned}$$

- Objective: choose $\mathbf{f}$ such that the conditional distribution of $\mathbf{y}_g \mid \mathbf{x}$ is close to the conditional distribution of the observed data $\mathbf{y} \mid \mathbf{x}$

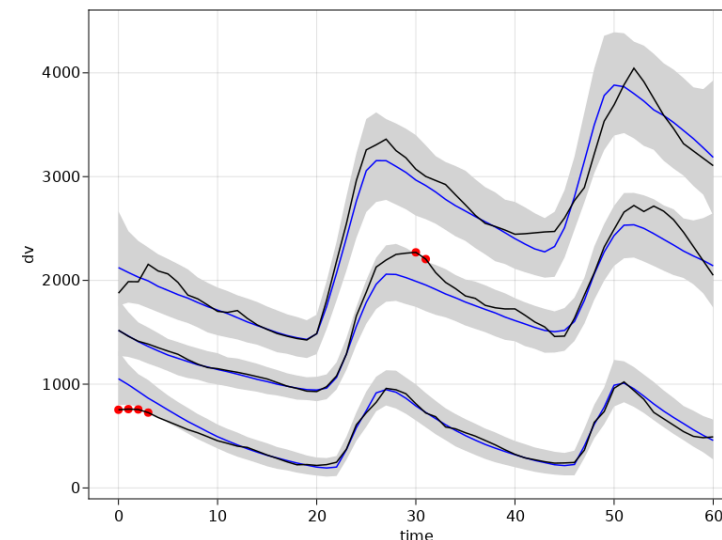


Population PK is machine learning!

- Definitions
 - $\boldsymbol{\eta}$: latent variables of dimension d and covariance matrix $\boldsymbol{\Omega}$
 - $\boldsymbol{x}$: observed covariates
 - $\mathbf{dv}$: observed response
 - $\mathbf{dv}_g$: generated/simulated/synthetic response

- Model

$$\begin{aligned}\mathbf{dv}_g &= \mathbf{f}_{\theta}(\boldsymbol{\eta}, \boldsymbol{x}) + \epsilon \\ \boldsymbol{\eta} &\sim \text{Normal}(0, \boldsymbol{\Omega}) \\ \epsilon_i &\sim \text{Normal}(0, \sigma^2)\end{aligned}$$



- Objective: choose $\mathbf{f}_{\theta}$ such that the conditional distribution of $\mathbf{dv}_g \mid \boldsymbol{x}$ is close to the conditional distribution of the observed data $\mathbf{dv} \mid \boldsymbol{x}$